INHA UNIVERSITY | ISE

SMART MOBILITY HUB

Smart Mobility Engineering Lab

ROS 2 + Python Project Assignment

Challenge: Build a miniature Songdo transportation station that detects passengers, predicts demand, coordinates vehicle arrivals, manages parking, routes traffic, and deploys an autonomous assistance robot. Every decision must produce an observable response and a reproducible test result.

Instructor	Prof. Mehdi Pirahandeh
Suggested duration	Seven weeks; relative milestones allow the instructor to set calendar dates.
Team size	Five students; recommended effort: 5–7 hours per student each week.
Platform	ROS 2 Jazzy (preferred) or Humble, Python `rclpy`; use one distribution consistently across the team and record exact installed versions.
Core deliverable	One integrated, simulation-first hub with eight functional scenarios plus a required safety/fault suite.
Physical extension	A tabletop robot, LEDs, and barrier servo when hardware is available.
Assessment	100 points: 80 team deliverables and 20 individual contribution.

Engineering boundary. This is an educational model, not a road-ready autonomous transport system. Use a closed test area. Software stop logic complements, but does not replace, an accessible hardware power stop during physical trials.

Document status. Complete assignment specification with interface contracts, starter patterns, scenario algorithms, and test procedures. Students implement the assessed system; starter fragments are not a completed ROS package.

Contents

1 Mission, learning outcomes, and minimum scope 2

2 Reference test map and coordinate conventions 3

3 Map graph, simulator, and safety envelope 4

4 ROS 2 architecture and Python node responsibilities 5

5 Topic contracts, rates, QoS, and frames 7

6 Custom messages and data validity 8

7 Services and actions: exact development contracts 10

8 Scenario S1: passenger detection and queue counting 14

9 Scenario S2: AI demand prediction 15

10 Scenario S3: bus arrival and docking 16

11 Scenario S4: parking availability and reservation 17

12 Scenario S5: congestion detection 18

13 Scenario S6: intelligent route replanning 19

14 Scenario S7: autonomous robot passenger escort 20

15 Scenario S8: integrated smart hub coordination 21

16 Scenario S9: negative tests, degraded operation, and emergency stop 22

17 Scenario communication matrix 23

18 Required algorithm pseudocode 24

19 Python starter patterns and package organization 26

20 Launch, inspection, and reproducible testing 28

21 Verification matrix and metric definitions 29

22 Optional physical implementation 31

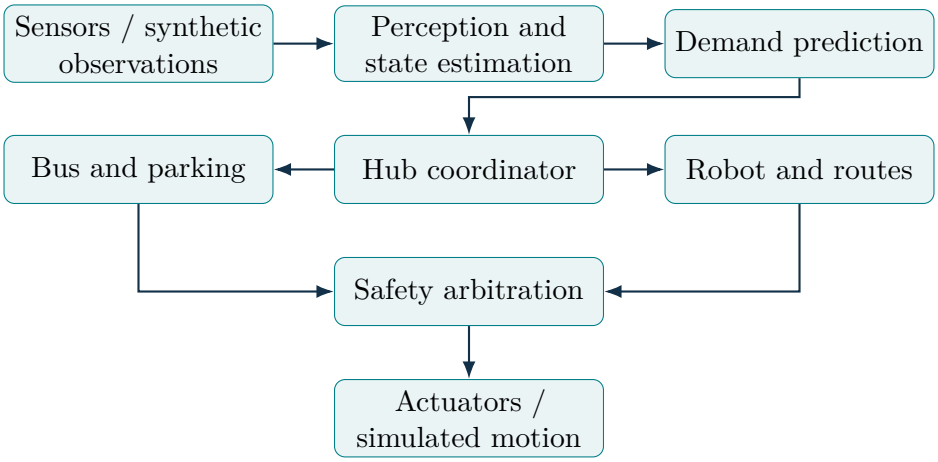
23 Seven-week work plan and five-person responsibilities 32

24 Submission, grading, demonstration, and references 33

1 Mission, learning outcomes, and minimum scope

1.1 The Songdo station story

A pedestrian approaches a local transport hub. A camera estimates the passenger queue. A prediction node forecasts near-term demand. A bus approaches a bay; cars search for parking; a small assistance robot guides a passenger. A temporary obstruction forces a route change. The system must coordinate these activities without conflicting movement at the crossing.



1.2 By the end, each student must be able to

1. Explain why a data stream uses a topic, a short transaction uses a service, and a cancellable mission uses an action.
2. Implement and diagnose Python ROS nodes with typed interfaces, timestamps, callback handling, and appropriate QoS.
3. Convert observations into validated states, forecast demand, and select a feasible route.
4. Integrate safe motion, mission cancellation, service failure handling, and stale-data detection.
5. Run controlled experiments and trace every reported result to recorded evidence.

1.3 Required versus extension work

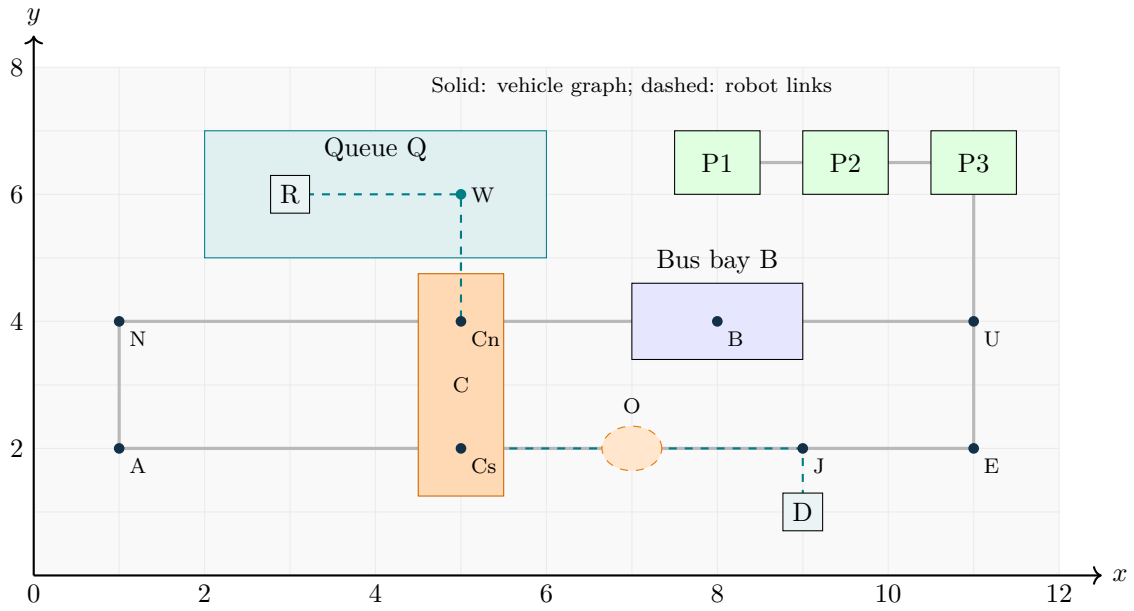
Area	Required baseline	Optional extension
Environment	Deterministic Python 2D simulator; coordinate map and RViz markers	Gazebo world and Nav2 integration
Perception	Synthetic detections plus one camera/video detection demonstration	Multi-camera tracking or sensor fusion
Prediction	Persistence baseline and one trained regression model	Uncertainty estimation or temporal neural model
Mobility	One bus, one assistance robot, three parking spaces	Several buses or cooperative robot fleet
Control	Waypoint motion with collision checks, crossing mutex, watchdog	Dynamics-aware planning and localization
Evidence	Eight scenarios, negative tests, metrics and replay	Physical replication of all scenarios

Do not spend the first weeks constructing a complex simulator. The assessed core is the integrated ROS behavior. Clearly label synthetic, video-based, simulated, and physical results.

2 Reference test map and coordinate conventions

2.1 Common map M1

The simulated site is a 12 m × 8 m rectangle. Origin is the southwest corner; $+x$ points east, $+y$ north, $+z$ up. Positions are in metres and yaw in radians. These are teaching coordinates, not a surveyed Songdo site. A physical 3 m × 2 m board uses scale 1:4; retain virtual coordinates in software and document the adapter conversion.



Feature	Coordinates / region	Test purpose
Entry A; exit E	$(1, 2)$; $(11, 2)$	Vehicle spawn and completion
Crossing C	$x \in [4.5, 5.5], y \in [1.25, 4.75]$	Exclusive occupancy by pedestrians, bus/car, or robot
Queue Q	$x \in [2, 6], y \in [5, 7]$	Passenger counting and demand prediction
Bus bay B	$(8, 4)$	Approach, docking, door interlock
Robot R; destination D	$(3, 6)$; $(9, 1)$	Escort mission and arrival tolerance
Parking	P1 $(8, 6.5)$; P2 $(9.5, 6.5)$; P3 $(11, 6.5)$	Occupancy, reservation, parking confirmation
Obstacle O	Centre $(7, 2)$; radius 0.35	Blocks edge Cs–J during rerouting

Zones override drawing appearance. Shared paths are conceptual. Pedestrians and all mobile agents must obey crossing ownership. Do not infer that paths are safe merely because their drawn centre lines do not intersect.

3 Map graph, simulator, and safety envelope

3.1 Graph definition

Use an undirected graph with edge length equal to Euclidean endpoint distance. Baseline vehicle edges are A–Cs, Cs–J, J–E, A–N, N–Cn, Cn–B, B–U, U–E. Add U–P3, P3–P2, P2–P1 for parking access. Robot edges are R–W, W–Cn, Cn–Cs, Cs–J, J–D, where W is $(5, 6)$. For robot detours, also allow Cn–B, B–U, U–E, E–J. Parking edges are car-only; the bus cannot enter them.

Crossing resource: any edge intersecting region C requires ownership before entry. This includes N–Cn, Cn–B, A–Cs, Cs–J, and Cn–Cs. Use segment–rectangle intersection, not just endpoint membership. Release ownership only when the full agent footprint has cleared C. Obstacle occupancy blocks an edge if the swept footprint intersects the obstacle; no route means a stopped vehicle and an explicit failure.

3.2 Minimum simulator loop

1. Store agent pose, velocity, mission, graph edge, footprint, and controller state.
2. At 20 Hz, apply only the most recent safe velocity, integrate a unicycle or waypoint kinematic model, and check swept motion for collisions.
3. Publish observed state separately from private ground truth. Detectors receive noisy observations; the metric evaluator alone reads ground truth.
4. At configured timestamps, inject arrivals, occlusion, blocked edges, sensor freezes, and stop events from a scenario file.
5. Publish RViz markers for agents, routes, zone ownership, obstacle footprints, parking state, and prediction text.

Use a fixed random seed, fixed time step, and reset all state between trials. The baseline can run on system ROS time with a monotonic timer for command watchdogs. If using a simulator that publishes `/clock`, enable `use_sim_time` consistently; never start a second clock publisher. During physical trials, watchdogs must continue to operate if simulation time pauses.

3.3 Lab parameters and acceptance targets

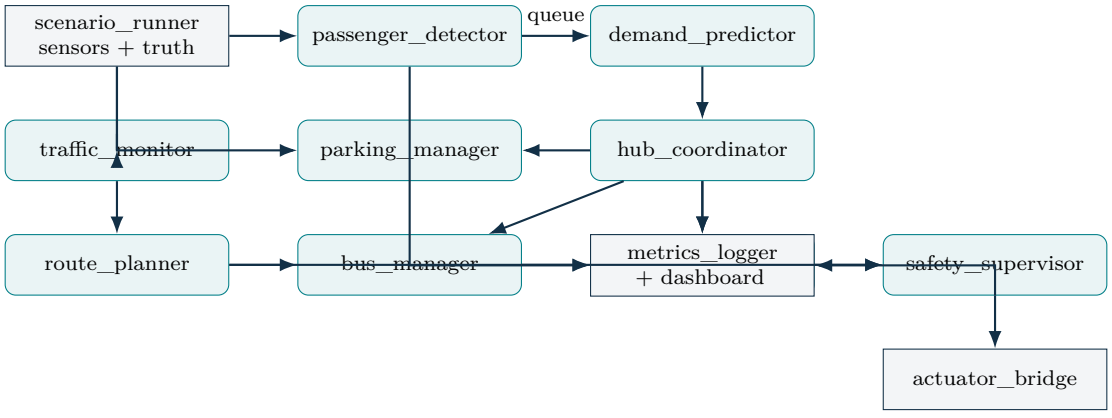
Parameter	Initial value	Meaning
Robot / bus speed caps	0.25/0.40 m/s	Virtual map rates; scaled physical motion must be recalibrated
Robot / vehicle radius	0.20/0.30 m	Circular baseline footprints
Added clearance	0.20 m	Clearance outside footprint boundary
Motion command timeout	0.50 s	Force zero velocity on stale command
Perception freshness	1.0 s	Older data becomes UNKNOWN
Command stop latency	≤ 0.20 s	Stop-event receipt to zero command in simulation
Arrival tolerance	0.20 m; 0.17 rad	Must hold for one second
Crossing clear debounce	2.0 s	Continuous clear observations before grant

These are assignment design targets, not certified safety limits. For hardware measure stopping distance at each allowed speed. Require obstacle distance at least $v\tau + v^2/(2a_{\min}) + m$, with measured deceleration $a_{\min} > 0$, latency τ, and margin m. Reduce speed if the available distance is insufficient.

4 ROS 2 architecture and Python node responsibilities

All application logic is written in Python using `rclpy`. The custom interface package uses `ament_cmake` and `rosidl`; generating message code does not require students to write C++ application nodes.

Node / owner	Responsibilities	Principal communication
`scenario_runner` / M5	Reset, seeded events, private truth, simulator stepping	Sensor/state publishers; test-only injection services
`passenger_detector` / M1	Detection, tracking, queue zone membership, confidence	Image subscriber; queue publisher
`demand_predictor` / M1	Sliding features, baseline, trained model, stale flags	Queue subscriber; demand publisher
`bus_manager` / M2	ETA and approach/dock/depart state machine	Vehicle subscriber; arrival publisher; bus action server
`parking_manager` / M3	Atomic reservations and expiry, sensor reconciliation	Occupancy subscriber; parking state publisher; reserve/release services
`traffic_monitor` / M3	Per-edge speed, density, blocked status	Vehicle subscriber; edge-state publisher
`route_planner` / M4	Dijkstra/A*, edge constraints and costs	Edge subscriber; compute-route service
`robot_manager` / M4	Mission action, path tracking, cancellation	Escort action server; route service client; candidate velocity publisher
`hub_coordinator` / M2	Dispatch policy and crossing resource arbitration	State subscribers; action clients; crossing service server
`safety_supervisor` / M5	Watchdog, obstacle clearance, latched stop	Command/state subscribers; final velocity publishers; reset service
`actuator_bridge` / M5	Simulated or real motors, barrier and LEDs	Final command subscribers; acknowledgment/status publishers
`metrics_logger` / M5	Run IDs, latency, violations, model errors	Observations, decisions, ground-truth subscribers
`dashboard_marker_publisher` / M5	Human-readable status and RViz map overlays	State/event subscribers; marker publisher; no control authority



4.1 Node implementation and verification contracts

The following table is the minimum node-level design record. “R1” means reliable, keep-last depth 1; “R10” means reliable depth 10; “sensor” means the ROS sensor-data profile. Rates are nominal and must be checked with timestamped evidence, not assumed from timer settings.

Node	Inputs	Outputs	Service / action role	Key parameters	Rate / QoS	Required evidence
`scenario_runner`	Scenario YAML, reset request	Images, observations, vehicles, private truth	ResetScenario server	seed, step, mode	20 Hz; R10	Repeatable seed and clean reset
`passenger_detector`	Image or synthetic tracks	`/hub/queue`	None	polygons, confidence, stale timeout	5 Hz; sensor in, R10 out	Counts, confidence, freeze test
`demand_predictor`	`/hub/queue`	`/hub/demand`	None	horizon, window, model path	1 Hz; R10	Held-out MAE/RMSE and latency
`bus_manager`	Vehicles, safety, crossing	Arrival and bus candidate command	DockBus server; route/crossing client	speed, pose tolerance, deadline	20 Hz control; R1 command	Feedback, cancel, dock error
`parking_manager`	Slot observations	Parking state	ReserveParking and ReleaseParking servers	TTL, debounce, stale timeout	2 Hz + change; R1T	Concurrent atomicity and expiry
`traffic_monitor`	Vehicle states	Edge states	None	capacities, window, thresholds	2 Hz; R10	Known-density and stale-edge tests
`route_planner`	Edge states, map	Route response/events	ComputeRoute server	cost weights, replan threshold	On request; reliable	Oracle cost and no-route tests
`robot_manager`	Odom, safety, crossing	Robot candidate command	EscortPassenger and NavigateTo servers; route/crossing client	speed, tolerances, timeout	20 Hz control; R1 command	Goal, feedback, cancel, arrival
`hub_coordinator`	Demand, parking, arrival, edge/crossing state	Decisions and crossing state	RequestCrossing server; action clients	thresholds, hysteresis, lease	10 Hz; R1T state	Single-owner and dispatch trace
`safety_supervisor`	Candidate commands, poses, hazards, e-stop	Final commands and safety state	ClearSafetyStop server	freshness, clearance, speed limits	20 Hz; R1/R1T	Stop latency and watchdog proof
`actuator_bridge`	Final commands, barrier/LED requests	Measured status and acknowledgments	None	hardware map, command timeout	20 Hz; R1	Command expiry and I/O trace
`metrics_logger`	All assessed states, events, private truth	CSV/JSON run record	None	output path, run ID	Event + sample; compatible QoS	Correlated bag and metric files
`dashboard_marker_publisher`	Hub states, routes, events	`/hub/markers` and display status	Read-only clients permitted	frame, marker lifetime	5 Hz; R10	RViz screenshot and no control publisher

4.2 Choose the interface from the behavior

- **Topic:** asynchronous repeated state, e.g., queue size or vehicle pose. Python APIs: `create_publisher`, `create_subscription`, `create_timer`.
- **Service:** bounded request/response, e.g., reserve a slot or compute a short graph route. APIs: `create_service`, `create_client`, `call_async`. Return errors explicitly; do not wait for physical completion inside a service.
- **Action:** ongoing task with goal, feedback, result, and cancellation, e.g., dock a bus or escort a passenger. APIs: `ActionServer`, `ActionClient`, `send_goal_async`.
- **Parameter:** configuration such as speed cap and confidence threshold. Validate ranges. Log changes that affect experiment results.

Only `safety_supervisor` publishes final motor velocity for each agent. Managers publish candidate commands. A dashboard must never bypass this arbitration.

5 Topic contracts, rates, QoS, and frames

All custom types below are in `smart_hub_interfaces`. Topic names are contracts; teams may namespace them with `/team_N` consistently.

Topic	Type	Rate / QoS	Publisher → subscribers
`/camera/image_raw`	`sensor_msgs/Image`	10 Hz; sensor	Camera/runner → detector
`/hub/queue`	`QueueState`	5 Hz; R10	Detector → predictor, coordinator, safety
`/hub/demand`	`DemandPrediction`	1 Hz; R10	Predictor → coordinator, dashboard
`/hub/vehicles`	`VehicleStateArray`	10 Hz; R10	Simulator/tracker → bus, traffic, safety
`/hub/bus_arrival`	`BusArrival`	2 Hz; R10	Bus manager → coordinator, display
`/parking/observed`	`SlotObservationArray`	5 Hz; R10	Sensors/runner → parking manager
`/parking/state`	`ParkingState`	2 Hz + change; R1T	Parking manager → coordinator, display
`/traffic/edges`	`EdgeStateArray`	2 Hz; R10	Monitor → planner, coordinator
`/crossing/state`	`CrossingState`	10 Hz; R1T	Coordinator → managers, safety
`/robot/odom`	`nav_msgs/Odometry`	20 Hz; R10	Simulator/driver → robot manager, safety
`/robot/cmd_vel_candidate`	`geometry_msgs/TwistStamped`	20 Hz; R1	Robot manager → safety
`/robot/cmd_vel`	`geometry_msgs/Twist`	20 Hz; R1	Safety → motor adapter
`/bus/cmd_vel_candidate`	`geometry_msgs/TwistStamped`	20 Hz; R1	Bus controller → safety
`/bus/cmd_vel`	`geometry_msgs/Twist`	20 Hz; R1	Safety → bus adapter
`/hub/estop`	`std_msgs/Bool`	Event + 10 Hz; R1T	Operator/runner → safety; true latches stop
`/hub/safety`	`SafetyState`	10 Hz; R1T	Safety → every mission manager
`/hub/events`	`HubEvent`	On change; R100	All decision nodes → logger
`/hub/markers`	`visualization_msgs/MarkerArray`	5 Hz; R10	Renderer → RViz

R means reliable and volatile; number is keep-last depth. T means transient-local durability. Use ROS sensor-data QoS for camera streams. Configure matching subscriptions and rosbag recording overrides where necessary. Reliable delivery is not a latency guarantee; every safety-relevant state still needs an age check. A retained sample is not necessarily fresh.

Frames: world `map`; robot `robot/odom` and `robot/base_link`; bus equivalents. Publish `map->robot/odom->robot/base_link` through one authority per transform. In the ideal simulator, map-to-odom may be identity. Stamp poses in their actual frames; do not mix camera pixels and map metres. Set candidate command frame to the agent's base frame. Gate freshness using both source timestamps and local receipt time, rejecting future timestamps outside a declared tolerance.

6 Custom messages and data validity

Create the files below. Repeated entries separated by a semicolon here become separate lines in the corresponding interface file. Use a `std_msgs/Headerheader` in every top-level state message, and document the observation timestamp rather than substituting publication time for old data.

Message	Required fields beyond the header	Why it is needed
`QueueState.msg`	`stringzone_id`; `uint32count`; `float32confidence`; `bool valid`; `boolcrossing_occupied`	Separates a valid zero count from missing perception and exposes crossing demand.
`DemandPrediction.msg`	`uint32horizon_sec`; `float32predicted_count`; `float32baseline_count`; `stringmodel_id`; `boolvalid`	Makes the forecast, baseline, horizon, and model traceable.
`VehicleState.msg`	`stringvehicle_id`; `stringkind`; `geometry_msgs/Pose2D pose`; `float32speed_mps`; `stringedge_id`; `boolvalid`	Provides one typed mobile-agent observation for routing and safety.
`VehicleStateArray.msg`	`VehicleState[]vehicles`	Publishes a synchronized traffic snapshot.
`BusArrival.msg`	`stringbus_id`; `stringstate`; `float32eta_sec`; `booleta_valid`; `booldoor_open`	Reports arrival progress and door interlock state.
`SlotObservation.msg`	`stringslot_id`; `booloccupied`; `float32confidence`; `boolvalid`	Keeps raw sensing distinct from reservation state.
`SlotObservationArray.msg`	`SlotObservation[]slots`	Carries one timestamped parking-sensor snapshot.
`ParkingSlot.msg`	`stringslot_id`; `stringstate`; `stringvehicle_id`; `string reservation_id`; `builtin_interfaces/Timeexpires_at`	Represents one slot's physical and administrative ownership.
`ParkingState.msg`	`ParkingSlot[]slots`; `uint32available_count`	Gives consumers an authoritative parking snapshot.
`EdgeState.msg`	`stringedge_id`; `float32mean_speed_mps`; `float32density_ratio`; `boolblocked`; `boolvalid`	Supplies congestion-aware planner costs and validity.
`EdgeStateArray.msg`	`EdgeState[]edges`	Publishes one version-consistent road graph state.
`CrossingState.msg`	`stringowner_id`; `stringtoken`; `stringstate`; `boolpedestrian_present`; `boolvalid`	Makes exclusive crossing ownership observable and auditable.
`SafetyState.msg`	`boolstop_latched`; `boolmotion_permitted`; `stringreason`	Broadcasts safety authority and the inhibition reason.
`HubEvent.msg`	`stringrun_id`; `stringscenario_id`; `stringentity_id`; `stringevent`; `stringreason`; `stringcorrelation_id`	Correlates decisions, transactions, actions, and metrics.

Nested records can omit a header if they are a synchronized snapshot under their array header. Otherwise give each record an observation timestamp; never silently bundle asynchronously stale records as fresh.

6.1 Required semantic rules

- Queue confidence lies in $[0, 1]$. Low-confidence or stale observations mean UNKNOWN, not zero people. A zero count requires a valid observation.

- Parking states are FREE, RESERVED, OCCUPIED, UNKNOWN. Only fresh FREE slots are allocatable. Reservation expiry never changes a physically occupied slot to FREE.
- Density ratio is $\min(1, n_e/K_e)$ with documented edge capacity K_e. Invalid speed must not become zero travel time.
- ETA is meaningful only when `eta_valid=true`; use `eta_sec=-1` otherwise. Units and invalid-value conventions belong in the README.
- Events report transitions, not every control tick. Include the reservation token or action goal ID in `correlation_id` to trace cause and effect.

7 Services and actions: exact development contracts

7.1 Short transactions

Implement each service in its owning node. The normal response target is 250 ms on the reference machine; client timeout is 1 s. Timeout does not prove the operation failed: use the same request ID when retrying a reservation.

```
# ReserveParking.srv -- server: parking_manager
string request_id
string vehicle_id
string preferred_slot     # empty means any permitted slot
float32 ttl_sec           # accept 5..120; default client value 30
---
bool success
string reservation_id
string slot_id
builtin_interfaces/Time expires_at
string reason

# ReleaseParking.srv -- server: parking_manager
string request_id
string reservation_id
string vehicle_id
---
bool success
string reason

# ComputeRoute.srv -- server: route_planner
string start_node
string goal_node
string agent_type        # robot, bus, car
string[] excluded_edges
---
bool success
string[] node_ids
geometry_msgs/Point[] waypoints
float32 estimated_cost_sec
string reason

# RequestCrossing.srv -- server: hub_coordinator
string requester_id
string operation         # ACQUIRE, RENEW, RELEASE
string token             # empty for ACQUIRE
---
bool granted
string token
string reason

# ResetScenario.srv -- server: scenario_runner
string scenario_id
uint32 seed
bool preserve_logs
---
bool success
string run_id
string reason

# ClearSafetyStop.srv -- server: safety_supervisor
string operator_id
string acknowledgement
---
bool success
string reason
```

`/parking/reserve_slot` and `/parking/release_slot` implement the first two services. `/route/compute`, `/crossing/request`, `/scenario/reset`, and `/safety/clear_stop` implement the remaining contracts. A denied crossing grant returns promptly; the caller retries with bounded backoff. Queue requests by arrival time, but pedestrians have priority. Renewal loss stops new entries; do not reassign an occupied crossing merely because a lease expires. Require observed clearance.

The ClearSafetyStop server rejects reset while hazards, stale critical sensors, an asserted stop input, or nonzero motion remain. A false stop message never automatically clears the latch. Reset authorizes readiness, not automatic mission restart. ResetScenario is test orchestration only: it cancels active goals, commands zero motion, restores the selected seeded world, clears administrative resources, and returns a new run identifier after all nodes acknowledge reset.

7.2 Cancellable missions

```
# DockBus.action -- /bus/dock; server: bus_manager
string bus_id
string bay_id             # B for baseline
float32 max_duration_sec
---
bool success
string final_state
string reason
---
string phase
float32 distance_remaining_m
float32 eta_sec

# EscortPassenger.action -- /robot/escort; server: robot_manager
string request_id
string passenger_id       # anonymous simulation identifier
string pickup_node        # R
string destination_node   # D
float32 max_duration_sec
---
bool success
string final_node
float32 travelled_distance_m
string reason
---
string phase
float32 progress          # 0..1; define across replans
float32 distance_remaining_m

# NavigateTo.action -- /robot/navigate_to; server: robot_manager
string request_id
string goal_node
float32 max_duration_sec
---
bool success
string final_node
float32 final_error_m
string reason
---
string phase
float32 progress
float32 distance_remaining_m
```

Feedback is published at 2 Hz. Reject invalid IDs, unreachable goals, nonpositive time limits, unsafe startup, and a second mission while busy. Set the goal deadline from acceptance time. Use a sufficiently long default, e.g., 240 s for the robot map. NavigateTo is the reusable motion primitive for unaccompanied robot repositioning; EscortPassenger wraps pickup, readiness, human-assistance policy, and one or more navigation phases. DockBus remains separate because its bay, door, and boarding interlocks are domain-specific.

Terminal semantics: call `succeed()` only after observed arrival and one-second stability. Call `canceled()` after accepting cancellation and confirming a zero command; report failure to stop separately. Call `abort()` for timeout, persistent blocked route, invalid localization, or a latched safety stop. Populate the result in every terminal path. An action accepted by the server is not proof of mission completion.

Cancellation versus protection: a cancel request uses the action protocol. A safety stop immediately overrides velocity independently of action callbacks; the action then reports its terminal state. Never rely on network cancellation alone for stopping.

7.3 Concurrency requirements

Do not put a long `while` loop or synchronous service call in a subscription or timer callback. Use short state-machine ticks, asynchronous futures, and explicit deadlines. If the execute callback remains active while subscriptions and cancellation must run, use a suitable reentrant callback group and a `MultiThreadedExecutor`; protect shared state. Keep reservation updates atomic under a lock or one serialized callback group. Avoid Python `asyncio.sleep` inside a ROS coroutine unless an actual asyncio event loop has been integrated.

8 Scenario S1: passenger detection and queue counting

Outcome: A person entering the queue is counted once, predicted demand changes, and a pedestrian in crossing C inhibits vehicle and robot entry.

8.1 Implementation map

Component	Required implementation
Inputs	`/camera/image_raw`; synthetic track observations for deterministic testing; calibrated queue and crossing polygons.
Publisher	`passenger_detector` publishes `/hub/queue` at 5 Hz.
Subscribers	Predictor consumes count history; coordinator consumes queue state; safety consumes crossing occupancy and validity.
Service / action	No service for continuous counting. Coordinator uses the crossing service for mobile agents; pedestrians cause occupancy-based inhibition.
Response	Queue display updates; crossing indicator becomes occupied; approaching agents stop before entry.

8.2 Student implementation steps

1. Establish a fixed camera view or prerecorded lab video. Detect people with an available pretrained detector or a documented simpler baseline. Record model name and threshold.
2. Associate detections over time using track IDs. Count tracks inside Q, not all bounding boxes and not cumulative entries. Use the foot-point of the bounding box for ground-plane mapping.
3. Use homography from at least four coplanar calibration points if converting image coordinates to the map. Document failure outside the calibrated floor plane.
4. Debounce track appearance/disappearance for 0.4 s; preserve identity across a short 0.5 s occlusion. Avoid keeping stale tracks indefinitely.
5. Publish count confidence and freshness. If the feed freezes, invalidate state after 1 s; do not continue publishing the old count with a fresh observation timestamp.
6. Set crossing occupied from actual region-C detections, independent of the queue count. Hold release until C has remained clear for 2 s.

8.3 Test script and pass criteria

Start empty for 5 s. Spawn three queue tracks at $t = 5$ s, two more at $t = 10$ s, remove one at $t = 15$ s. Add a crossing pedestrian at $t = 20$ s for 4 s. Freeze the camera at $t = 30$ s. Run for 40 s.

- Synthetic stable counts must be $0, 3, 5, 4$ after the debounce interval; repeated frames must not inflate the count.
- A 0.3 s occlusion must not create a new passenger; invalid state must appear within 1.2 s of feed freeze.
- Crossing blocks new entry while occupied and during the 2 s clear debounce. Record zero crossing conflicts.
- On at least 100 manually annotated video frames, report count MAE, precision/recall if detection labels exist, and performance under partial occlusion. The initial count-MAE target is ≤ 1 person; report failures honestly.

Evidence: annotated before/during/after frames, count-versus-truth plot, source timestamps, and a video showing the inhibited motion.

9 Scenario S2: AI demand prediction

Outcome: Forecast the queue 30 seconds ahead and use the forecast to request service. Prediction cannot override crossing or motion safety.

9.1 ROS development

`demand_predictor` subscribes to `/hub/queue`, maintains 60 s of history, and publishes `/hub/demand` at 1 Hz. `hub_coordinator` subscribes to demand and bus/robot state. It is an action client for `/bus/dock` and `/robot/escort`. No inference service is required for the streaming baseline.

9.2 Prediction specification

At one-second samples, define q_t as the latest valid queue count. The persistence baseline is $\widehat{q}^{(0)}_{t+30} = q_t$. Train a ridge regressor or another small documented regressor using

$$x_t = [q_t,\ \overline{q}_{t-9:t},\ q_t - q_{t-10},\ a_{t-29:t},\ d_{t-29:t}], \qquad \widehat{q}_{t+30} = \max(0, f_\theta(x_t)),$$

where a and d are observed queue entry and exit counts over the last 30 s. Derive them from tracks or a clearly labelled synthetic event source. If only queue counts exist, use documented proxy features and explain the lost information.

Generate at least 30 independent six-minute demand episodes using varied arrival rates and boarding events. Split whole episodes 60/20/20 into training/validation/test sets before making overlapping windows. Keep unseen test seeds separate; fitting normalization and selecting hyperparameters use training/validation only. No future arrival schedule, future queue, or hidden ground truth may be an online feature.

9.3 Decision implementation

1. Warm up until all features exist; publish `valid=false` during warm-up or stale input. Display "prediction unavailable" explicitly.
2. If $\widehat{q}_{t+30} \geq 6$ for three consecutive valid predictions and the bus is available, request one DockBus action.
3. Keep an outstanding-goal flag, goal ID, and a 30 s request cooldown; avoid repeated bus requests every second.
4. Clear demand escalation only after prediction is below 4 for five consecutive samples. This hysteresis avoids rapid switching.
5. If prediction is unavailable, fall back to a documented observed-queue rule, e.g., $q_t \geq 6$ with fresh observations. Mark the decision reason as FALLBACK.

9.4 Tests

Use a rising-demand episode, a steady queue, and a noisy sequence around threshold 6. Include a feed gap after warm-up. Report test MAE, RMSE, inference latency, and baseline comparison. Initial target: inference p95 below 100 ms and trained-model MAE no worse than persistence on the held-out set. A worse model must remain visible in the report; demonstrate that the baseline can be selected safely.

Require exactly one dispatch during each outstanding mission, no dispatch from an invalid forecast, and recorded fallback behavior. Compare service waiting time under reactive and predictive policies on identical episodes. Prediction accuracy alone does not establish transport benefit.

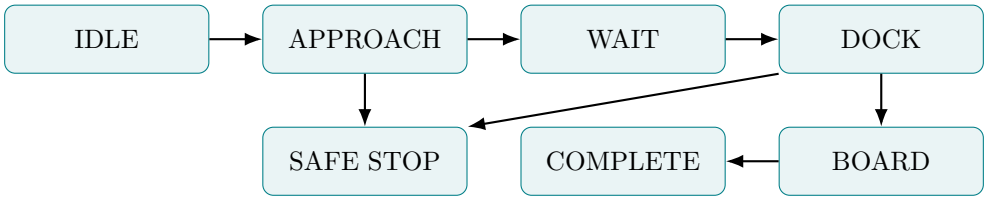
10 Scenario S3: bus arrival and docking

Outcome: A bus announces arrival, acquires crossing permission, docks at B, opens its door only while stopped, and completes a cancellable action.

10.1 ROS development

`bus_manager` subscribes to vehicle state and safety state; publishes `/hub/bus_arrival` and candidate velocity; serves `/bus/dock`. It calls `/route/compute` and `/crossing/request` asynchronously. The coordinator is the DockBus action client. The motor bridge subscribes only to safe final commands. Use reliable depth-1 `std_msgs/Bool` topics `/bus/door_request` (manager to safety/bridge) and `/bus/door_open` (measured bridge status to manager/safety). Gate opening on measured stopped state, and gate propulsion on confirmed closed doors.

10.2 State machine



WAIT is entered when crossing permission or a clear bay is unavailable; it may be skipped when all preconditions hold. A stop or failure is possible from every moving state.

1. Start the bus at A. Plan A–N–Cn–B, unless a cheaper allowed route is selected. Compute remaining route length, not straight-line distance through obstacles.
2. Estimate ETA from remaining travel length divided by valid smoothed speed, plus a declared dwell/wait estimate. If stopped without a credible wait estimate, set ETA invalid.
3. Acquire the crossing before the swept footprint enters C. Stop outside C if denied. Renew permission while crossing; release only after clearance.
4. Reduce speed on final approach. Declare docked only after position/yaw tolerance holds for 1 s and measured speed is below 0.02 m/s.
5. Open simulated doors for an 8 s boarding interval only when docked. Keep motion zero while doors are open; close doors before any departure command. The simulator boards up to four waiting passengers, removing their queue tracks and logging individual boarding times; leave excess passengers waiting.
6. Complete DockBus after boarding and door closure. Departure is a separate state transition or extension action, not an unreported continuation of the completed goal.

10.3 Tests and evidence

Run normal docking, pedestrian occupancy before entry, blocked bay, cancellation during approach, and stale bus pose. Target 10/10 nominal dockings without collision; position error ≤ 0.20 m; no open-door movement; cancellation produces zero command within 0.5 s of acceptance. For a moving, unobstructed bus, report ETA absolute error with a target ≤ 3 s at 5 m remaining. Exclude invalid ETA intervals from the error statistic but report their duration.

Save the action goal/feedback/result, path trace, crossing grant/release events, door state, measured speed, and final pose error. A screenshot of a parked bus alone is insufficient.

11 Scenario S4: parking availability and reservation

Outcome: Two simultaneous parking requests cannot acquire the same slot. Unknown or occupied slots are never advertised as free.

11.1 ROS development

`parking_manager` subscribes to `/parking/observed`, publishes `/parking/state`, and serves reserve/release services. Coordinator or a car client sends reservations. A simulated car movement may use a separate action as an extension; do not make ReserveParking wait for the car to arrive.

11.2 Implementation algorithm

1. Initialize every slot as UNKNOWN until a fresh observation arrives. Accept occupancy changes after 0.5 s of stable valid sensor readings.
2. Under one lock, validate request, check its idempotency key, reject invalid TTL, filter fresh FREE slots, choose the preferred feasible slot or nearest available, and commit a unique reservation token.
3. Record vehicle ID and expiry. Return the original result for a repeated request ID with the identical payload; reject reuse of the key with a different payload.
4. Exclude RESERVED slots from available count. A valid occupied observation changes the slot to OCCUPIED while preserving reservation ownership metadata for reconciliation.
5. On reservation timeout, release only the administrative reservation. The resulting display is FREE only if current sensing confirms empty; otherwise OCCUPIED or UNKNOWN.
6. Require the matching token and vehicle ID for release. A release never overrides a sensor showing an occupied slot. Use an expiry timer; clients do not own the slot state.

11.3 Exact test sequence

Initialize P1 OCCUPIED, P2 FREE, P3 FREE. At $t = 5$ s send requests from C1 and C2 concurrently, both preferring P2. One gets P2; the other gets P3 or an explicit rejection under a documented strict-preference policy. At $t = 6$ s repeat C1's request ID. At $t = 8$ s confirm C1 occupying its slot. At $t = 10$ s disconnect P3's sensor. At $t = 40$ s expire remaining reservations.

- Duplicate request returns the same token and does not allocate a second slot.
- Every run preserves at most one active owner per slot and one slot per vehicle.
- Wrong-token release is rejected; expired tokens cannot free a new owner's reservation.
- After expiry, an occupied or stale slot remains unavailable. All-full requests return a reason and never hang.
- Over 20 synchronized request-pair trials, require zero double allocations and p95 service response under 250 ms on the reference machine.

Physical response: green FREE, amber RESERVED, red OCCUPIED, blinking amber UNKNOWN. Open a barrier only for a valid owner with a fresh clear entrance sensor. A dashboard label alone does not demonstrate barrier logic; log the command and acknowledgment. Use reliable depth-1 `std_msgs/Bool` topics `/parking/barrier_request` and `/parking/barrier_open`; true means open. The bridge applies occupancy interlocks and publishes observed status; a request is not an acknowledgment.

12 Scenario S5: congestion detection

Outcome: The hub converts timestamped vehicle observations into valid per-edge speed, density, congestion, and blockage states without confusing stale data with an empty road.

12.1 Map setup and initial conditions

Use vehicle edges A–Cs, Cs–J, J–E and the upper detour. Begin with one car on Cs–J at 0.35 m/s and no obstruction. Set edge capacities from length and the declared safe headway. At $t = 20$ s add three slow vehicles to the lower corridor; at $t = 45$ s freeze one source; at $t = 60$ s place O across Cs–J.

12.2 ROS development and algorithm

`traffic_monitor` subscribes to `/hub/vehicles` at 10 Hz and publishes `/traffic/edges` at 2 Hz. Assign a vehicle to an edge using its reported edge ID only after validating that its pose lies within the edge corridor. In a five-second sliding window compute robust mean speed, vehicle count n_e, density $\rho_e = \min(1, n_e/K_e)$, and observation age. Mark congestion when $\rho_e \geq 0.70$ or valid mean speed remains below 40% of free-flow speed for three publications. Mark a geometrically obstructed edge blocked immediately. A stale edge is invalid, not uncongested.

12.3 Event sequence, failures, and acceptance

The dashboard must show NORMAL, CONGESTED, BLOCKED, or UNKNOWN for each edge. Test incorrect edge IDs, a pose outside the claimed corridor, zero vehicles, frozen timestamps, and a sensor recovery. The monitor must reject impossible samples, change Cs–J to CONGESTED within 2 s of the sustained threshold, change it to BLOCKED within one publication after O is observed, and publish UNKNOWN within 1.2 s of loss of all valid observations. Require no false congestion in ten free-flow runs.

Metrics and evidence: congestion precision/recall against scenario labels, detection delay, false-alarm count, valid-data fraction, and edge-state plots. Record the vehicle input and edge output in the same bag. The route planner consumes this state in S6, but S5 passes independently without commanding motion.

13 Scenario S6: intelligent route replanning

Outcome: Route selection changes when a lower road is blocked or costly, and the planner explicitly fails when no permitted route exists.

13.1 ROS development

`traffic_monitor` consumes vehicle states and publishes edge states. `route_planner` subscribes to those states and serves ComputeRoute. Bus/robot managers call it, then execute the route through their mission actions. A route computation is a short service; route execution is the long-running task.

13.2 Cost and replanning rule

For edge e, length L_e, density ratio ρ_e, expected crossing wait w_e, and bounded nominal/smoothed speed v_e, use

$$c_e = \frac{L_e}{\max(v_e, 0.05)}(1 + 2\rho_e) + w_e.$$

All costs are seconds and nonnegative. Remove blocked edges and edges forbidden to the agent; do not merely assign a small penalty. If a critical edge observation is stale, treat it as blocked in the baseline. Dijkstra is sufficient for this map. If using A*, prove the heuristic cannot overestimate remaining cost.

Replan immediately for an invalid current route. For congestion-only changes, replan at most once per 2 s and only if the alternative cost is at least 15% lower. Preserve the crossing lock while physically inside C; route changes do not teleport the agent or transfer ownership.

13.3 Implementation and tests

1. Baseline car route A–Cs–J–E has length 10 m; upper detour A–N–Cn–B–U–E has length 14 m. Confirm these costs at equal speed and zero density/wait.
2. Insert O at $(7, 2)$, blocking Cs–J. Plan through the upper road. Check all returned edges against the current blocked set before executing.
3. Raise density on the lower road without blocking it. Verify the chosen path minimizes the stated cost, not simply geometric distance.
4. Block both Cs–J and Cn–B. For a vehicle from A to E, return `success=false` with NO_ROUTE; managers stop and wait or terminate according to their action deadline.
5. Restore the road. Require a new explicit route validation before movement resumes. Do not execute a cached path solely because it was previously valid.

For 20 seeded edge-state configurations compare the planner to an independent exhaustive simple-path enumerator on this small graph. Require valid start/end nodes, allowed edge continuity, no blocked edges, and cost equality within 10^{-6} relative tolerance. Planning latency target: p95 below 250 ms.

Evidence: before/after maps with edge costs, old/new path, reroute reason, compute latency, and travel time on paired seeds. Do not claim congestion improvement from one favorable screenshot.

14 Scenario S7: autonomous robot passenger escort

Outcome: A robot accepts a passenger mission, reaches pickup, waits for readiness, escorts along a safe route, and responds correctly to obstacle, cancellation, and timeout.

14.1 ROS development

`robot_manager` serves EscortPassenger and NavigateTo, subscribes to odometry/safety/crossing state, publishes candidate velocity, and calls ComputeRoute and RequestCrossing. Add `/robot/confirm_ready` using `std_srvs/Trigger`; accept readiness only while waiting at pickup. A simulated button or dashboard is the service client. Report out-of-state calls as failures.

14.2 Required state machine

IDLE → TO_PICKUP → WAIT_READY → ESCORT → ARRIVED.

BLOCKED and WAIT_CROSSING pause motion. Terminal outcomes are CANCELLED, FAILED, or SAFE_STOP; successful completion is ARRIVED.

1. Validate IDs and request deduplication. Obtain a route to pickup and then destination; compute a feasible mission deadline.
2. At pickup, stop and prompt the passenger. Wait up to 20 s for readiness; timeout aborts without movement.
3. Track waypoints using heading error and bounded speed. Stop forward motion for large heading error; cap angular rate and linear speed with parameters.
4. Before C, acquire a crossing grant. Maintain minimum obstacle clearance and use odometry feedback rather than elapsed time alone to infer progress.
5. During ESCORT, inject O. Stop before the blocked edge, request a new route through Cn–B–U–E–J–D if reachable from the current safe waypoint, and resume after validation. If already past the branch, return to a safe permitted waypoint only if reversal is collision-free.
6. When within destination tolerance for 1 s, publish arrival, stop, and succeed. On any terminal outcome, release resources only when physical clearance is confirmed.

14.3 Interaction and testing

The minimum interaction is an explicit ready button and progress display; follow-person vision is an extension. Do not claim person-following from waypoint navigation. Use anonymous passenger IDs; no face recognition is required.

Run: normal escort; no readiness confirmation; obstacle before crossing; cancel while moving; a second goal while busy; unreachable destination; odometry freeze. Require 10/10 nominal successes, no obstacle contact or crossing conflict, destination error ≤ 0.20 m, and feedback at least 2 Hz except documented scheduling jitter. Progress may decrease after a detour only if its definition explicitly allows it; remaining distance must reflect the new route.

For cancellation, measure both request-to-acceptance and acceptance-to-zero-command. For physical work also measure actual stop time and distance. A zero command is not evidence that the chassis already stopped.

15 Scenario S8: integrated smart hub coordination

Outcome: Passenger demand, a docking bus, parking requests, and an escort robot coexist without double reservations, conflicting crossing grants, or repeated dispatch.

15.1 Integration sequence

Use the same map, one bus and one robot. Spawn synthetic cars only on valid edges. Test scripts define inputs, not the required internal output states.

Time	Input / event	Expected observation
0–60 s	Warm-up queue history; P1 occupied; P2/P3 free	Forecast becomes valid only after sufficient history
60 s	Passenger arrivals rise; observed queue reaches 7	Forecast/observed fallback requests one bus mission
65 s	C1/C2 request parking simultaneously	Distinct reservations or explicit rejection
70 s	Escort request R to D	Accepted if ready and idle; pickup prompt appears
75 s	Passenger readiness confirmation	Robot requests crossing when needed
80–85 s	Pedestrian occupies C	No new bus/car/robot entry; existing movement stops safely as needed
90 s	Lower-road obstruction inserted	Stopped approach and feasible detour or explicit NO_ROUTE
120 s	Remove obstruction	New route checked; no spontaneous restart after a latched stop
180–300 s	Demand subsides; missions finish	Resources released after clearance; complete run summary

Timing is a reproducible stimulus schedule. A vehicle that has not yet reached the crossing is not forced there at a fixed timestamp. Make outcome assertions conditional on observed state and bound the total mission duration.

15.2 Coordinator policy

Pedestrians inhibit new crossing grants. Among waiting mobile agents, use FIFO after safe clearance. Retain one owner, token, acquisition time, and observed occupancy. A waiting robot cannot overwrite a bus token. A lost owner heartbeat causes a hold until the crossing is observed empty. Safety has priority over every optimization objective.

Demand dispatch, parking reservations, and robot missions have independent correlation IDs. Service timeout must not create duplicate reservations; a late action response must not trigger another action. Centralize retry/backoff logic and bound request queues.

15.3 Acceptance

Run ten seeded rush-hour episodes. Require zero collisions, zero overlapping crossing ownership, zero double allocations, and zero duplicate bus dispatches while a prior mission is active. Report completion rate, p50/p95 queue wait, average vehicle travel time, crossing waiting time, and robot mission success. Compare predictive and reactive dispatch on the identical ten seeds. Calculate safety violations separately; a lower average waiting time cannot compensate for a violation.

Evidence: synchronized overhead video, ROS graph, event timeline, reservations table, route changes, per-run metric CSV, and a written explanation of one difficult integration failure and its correction.

16 Scenario S9: negative tests, degraded operation, and emergency stop

Outcome: Faults produce an explicit safe state, bounded stopping behavior, and deliberate recovery. Unknown state never becomes permission to move.

16.1 Fault-to-response map

Injected fault	Required detection / ROS mechanism	Required response
Camera silence or frozen source stamp	Local receipt timer and source age in queue state	UNKNOWN crossing; deny new entry; forecast invalid
Robot odometry gap $>$ 1 s	Safety subscriber watchdog	Zero robot command; abort current escort
Candidate command gap > 0.5 s	Independent safety timer	Zero command even if last command was nonzero
Planner unavailable	Async service timeout after 1 s	Stop; retry at most twice; explicit failure
Duplicate reserve request	Request-ID cache and payload comparison	Same result; no extra allocation
Parking sensor stale	Slot observation age	UNKNOWN; unavailable for allocation
Emergency stop true	Latched safety input	Immediate zero outputs; actions abort; doors/barrier use defined safe policy
Crossing owner disappears	Owner heartbeat and zone observation	Hold crossing; reassign only after proven clearance

16.2 Implementation requirements

Run the safety tick at 20 Hz with a monotonic receipt clock. Evaluate: stop latch, critical input freshness, command freshness, obstacle clearance, door interlock, speed bounds, and crossing permission. On failure publish zero velocity and a reason. The adapter must also stop when final commands expire, so a crashed supervisor cannot leave a latched nonzero motor command.

Use separate stop policies for the bus and robot, but shared crossing uncertainty stops entry by both. Never close a physical barrier on an occupied passage; a "safe" actuator state must be defined per device. For a tabletop demo, disable propulsion and hold the barrier until the obstruction is cleared.

16.3 Recovery test

At $t = 10$ s while the robot moves, assert stop. Verify zero output within 0.20 s of supervisor receipt. Send false at $t = 12$ s: motion stays inhibited. Request reset while a stale sensor remains: reject it. Restore fresh sensors and clear hazards; request reset again: accept it, but keep the robot stopped until a new goal arrives.

Repeat each fault five times at different mission phases. Report zero-command latency and actual simulator velocity separately. No live hardware fault injection may disable the independent manual stop. Use process termination tests in simulation first.

Pass conditions: all fault cases have a named logged reason; no silent continuation; no automatic restart after a latched stop; a supervisor crash is caught by the adapter watchdog. State explicitly which targets are met only in simulation.

17 Scenario communication matrix

Every scenario must expose the following ROS 2 communication explicitly. Topics carry changing state, services complete bounded transactions, actions represent cancellable missions, and parameters configure behavior without becoming runtime event channels.

Scenario	Topics	Services	Actions	Why this division is required
S1 Passenger detection	`/camera/image_raw`, `/hub/queue`, `/crossing/state`, `/hub/markers`	None	None	Counts and occupancy change continuously; no request or mission is created.
S2 AI demand prediction	`/hub/queue`, `/hub/demand`, `/hub/events`	None	DockBus client only when dispatch policy fires	Prediction is a stream; dispatch is a long-running cancellable consequence.
S3 Bus arrival/-docking	`/hub/vehicles`, `/hub/bus_arrival`, candidate/final velocity, door state, safety	`/route/compute`, `/crossing/request`	`/bus/dock`	Planning and lock requests are bounded; docking has progress, timeout, cancel, and result.
S4 Parking	`/parking/observed`, `/parking/state`, barrier request/status	`/parking/reserve_slot`, `/parking/release_slot`	None in baseline	Reservation is atomic request/response; occupancy remains a live state stream.
S5 Congestion	`/hub/vehicles`, `/traffic/edges`, `/hub/markers`	None	None	Edge conditions are repeated estimates consumed by several nodes.
S6 Route replanning	`/traffic/edges`, `/hub/events`	`/route/compute`	NavigateTo, DockBus, or EscortPassenger executes the returned route	Route calculation is short; physical traversal is long-running and cancellable.
S7 Robot escort	Odom, candidate/final velocity, crossing and safety state	`/route/compute`, `/crossing/request`, `/robot/confirm_ready`	`/robot/escort`; internal/reusable `/robot/navigate_to`	Escort has phases, feedback, cancellation, and a terminal result.
S8 Integrated hub	All assessed state/event topics	Parking, route, crossing, and reset services	DockBus and EscortPassenger clients	Correlation IDs keep simultaneous transactions and missions independent.
S9 Safety/negative	`/hub/estop`, `/hub/safety`, commands, state/event topics	`/safety/clear_stop`, `/scenario/reset`	Active actions abort or cancel	E-stop is asynchronous broadcast; deliberate recovery is validated request/response.

Scenario parameters. At minimum expose `queue_confidence_min`, `prediction_horizon_sec`, `dispatch_high`, `dispatch_low`, `reservation_ttl_sec`, `congestion_density_threshold`, `replan_improvement_ratio`, `robot_speed_limit`, `bus_speed_limit`, `crossing_clear_sec`, and `command_timeout_sec`. Reject invalid ranges at startup and record the final parameter dump with every run.

18 Required algorithm pseudocode

The pseudocode defines behavior, not a required class layout. Students may improve an algorithm while preserving interface contracts, safety invariants, and testable outputs.

18.1 Queue detection and demand prediction

```
QUEUE_TICK(frame, source_stamp):
  if source_stamp is stale or frame invalid: publish QueueState(valid=false); return
  detections <- PERSON_DETECTOR(frame)
  tracks <- TRACK_AND_ASSOCIATE(detections, previous_tracks)
  count <- number of confirmed track foot-points inside polygon Q
  crossing <- any confirmed track foot-point inside polygon C
  publish QueueState(count, confidence, valid=true, crossing_occupied=crossing)

PREDICT_TICK(queue_history):
  if history incomplete or newest sample stale: publish prediction(valid=false); return
  features <- [current count, rolling mean, trend, arrivals, departures]
  baseline <- current count
  prediction <- max(0, model(features))
  publish DemandPrediction(prediction, baseline, horizon=30, model_id)
```

18.2 Atomic parking reservation

```
RESERVE(request):
  lock state
  if request_id seen: return cached result if payload matches, else reject
  candidates <- fresh slots whose state is FREE and allowed for vehicle
  if preferred slot is feasible: chosen <- preferred else chosen <- nearest candidate
  if no candidate: cache and return FULL
  atomically set chosen to RESERVED with unique token and expiry
  cache request_id -> result; unlock; publish ParkingState; return result
```

18.3 Dijkstra/A* and congestion-aware weighting

```
EDGE_COST(edge, agent):
  if blocked, stale-critical, or forbidden(agent): return INFINITY
  travel <- length / max(smoothed_speed, 0.05)
  return travel * (1 + 2 * density_ratio) + expected_crossing_wait

COMPUTE_ROUTE(start, goal, agent):
  initialize distance[start]=0, all others=INFINITY; push start
  while priority queue not empty:
    u <- pop node with minimum cost
    if u == goal: return reconstructed path and cost
    for each allowed neighbor v: relax(u, v, EDGE_COST(u-v, agent))
  return NO_ROUTE

REPLAN(current_route):
  if any current edge invalid or blocked: replan immediately
  else if cooldown elapsed and new_cost <= 0.85 * remaining_cost: replace route
```

18.4 Crossing mutex arbitration

```
REQUEST_CROSSING(requester, operation, token):
  refresh observed occupancy and owner heartbeat
```

```
  if operation == RELEASE and token matches and crossing proven clear: clear owner
  if operation == RENEW and token matches: extend lease; grant
  if operation == ACQUIRE:
    if pedestrian present, state unknown, or owner exists: deny with reason
    if crossing continuously clear for clear_debounce: assign one token; grant
  never transfer ownership while any footprint occupies C
```

18.5 Robot escort action server

```
ESCORT(goal):
  validate goal, safety, exclusivity, route, and deadline; otherwise reject
  navigate to pickup; publish feedback; honor cancel and safety every control tick
  stop; wait at most 20 s for explicit passenger-ready confirmation
  while destination not stable within tolerance:
    if cancel: stop, confirm zero, release safe resources, return CANCELED
    if safety stop or stale odometry: stop and ABORT
    if edge blocked: stop, ComputeRoute from current safe node; abort if no route
    acquire/renew crossing token before entering C; follow validated waypoints
  stop for 1 s, release after clearance, return SUCCEEDED
```

18.6 Bus docking state machine

```
IDLE -> APPROACH after valid DockBus goal and route
APPROACH -> WAIT when bay/crossing unavailable; -> DOCK near bay
DOCK -> BOARD only after pose tolerance held 1 s and measured speed < 0.02 m/s
BOARD: command door open, hold zero motion for dwell, then confirm door closed
BOARD -> COMPLETE with action success
ANY MOVING STATE -> SAFE_STOP on cancel, stale pose, timeout, or safety latch
```

18.7 Safety supervisor watchdog

```
SAFETY_TICK(monotonic_now):
  reasons <- asserted e-stop, latched stop, stale pose/sensor/command,
             insufficient clearance, crossing violation, door interlock, overspeed
  if reasons nonempty: latch when required; publish zero final commands and reasons
  else: clamp fresh candidate commands to configured limits and publish finals
ACTUATOR_TICK: if final command expired, command zero independently
CLEAR_STOP: accept only when e-stop false, hazards clear, data fresh, motion zero
```

19 Python starter patterns and package organization

19.1 Recommended workspace

```
smart_hub_ws/src/
  smart_hub_interfaces/     # ament_cmake; msg/, srv/, action/
  smart_hub/                # ament_python; application nodes
    smart_hub/
      passenger_detector.py  demand_predictor.py
      bus_manager.py         parking_manager.py
      traffic_monitor.py     route_planner.py
      robot_manager.py       hub_coordinator.py
      safety_supervisor.py   scenario_runner.py
      metrics_logger.py      actuator_bridge.py
    launch/hub.launch.py
    config/map.yaml
    config/hub.yaml
    scenarios/S1.yaml ... S9.yaml
    tests/
    package.xml
    setup.py
    setup.cfg
```

Declare `rclpy`, standard interface packages, and custom-interface dependencies in package metadata. Register console scripts and install launch/config/scenario resources. Interface CMake must call `rosidl_generate_interfaces` with all files and dependencies, export runtime dependencies, and use `ament_package()`. Include `rosidl_default_generators`, `rosidl_default_runtime`, and membership in `rosidl_interface_packages` in its package metadata.

19.2 Minimal timestamped queue publisher

This standalone teaching example publishes a synthetic count; replace the constant with valid detector output. It does not implement crossing detection.

```
import rclpy
from rclpy.node import Node
from smart_hub_interfaces.msg import QueueState

class QueuePublisher(Node):
    def __init__(self):
        super().__init__('queue_example')
        self.pub = self.create_publisher(QueueState, '/hub/queue', 10)
        self.timer = self.create_timer(0.2, self.tick)

    def tick(self):
        msg = QueueState()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'map'
        msg.zone_id = 'Q'
        msg.count = 3
        msg.confidence = 1.0
        msg.valid = True
        msg.crossing_occupied = False
        self.pub.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = QueuePublisher()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()
```

19.3 Async service client pattern

The following methods belong inside a Node subclass. Create the client in `__init__`; call `request_route` from a short callback. Add a monotonic deadline timer, late-response handling, and bounded retries in the assessed implementation.

```
from smart_hub_interfaces.srv import ComputeRoute

# In __init__:
# self.route_client = self.create_client(ComputeRoute, '/route/compute')
# self.pending_route = None

def request_route(self):
    if self.pending_route is not None:
        return
    if not self.route_client.service_is_ready():
        self.get_logger().warning('Planner unavailable; stay stopped')
        return
    req = ComputeRoute.Request()
    req.start_node = 'R'
    req.goal_node = 'D'
    req.agent_type = 'robot'
    req.excluded_edges = []
    self.pending_route = self.route_client.call_async(req)
    self.pending_route.add_done_callback(self.on_route)

def on_route(self, future):
    try:
        result = future.result()
        if not result.success:
            self.get_logger().warning(result.reason)
            return
        self.get_logger().info('Route: ' + ','.join(result.node_ids))
        # Validate route version and current hazards before execution.
    except Exception as exc:
        self.get_logger().error(str(exc))
    finally:
        self.pending_route = None
```

19.4 Action lifecycle implementation checklist

1. Construct `ActionServer(self,EscortPassenger,'/robot/escort',...)` with goal, cancel, and execute callbacks, using the selected callback group.
2. In the goal callback validate the mission and reserve the single active-goal slot atomically. Return ACCEPT or REJECT.
3. In the execute path advance the state machine, publish feedback, and observe cancellation and safety without blocking unrelated callbacks.
4. On cancel, command stop first, confirm stopped state within a bound, then finish with the correct terminal status. Clear busy state in all exit paths.
5. In the coordinator, call `send_goal_async`; inspect acceptance, register feedback, and call `get_result_async`. Handle rejection separately from execution failure.

Use the official Python action tutorial as the API reference, then implement the lab-specific state machine. Do not copy a numerical tutorial's blocking loop into the movement controller.

20 Launch, inspection, and reproducible testing

20.1 Build and inspect

The commands assume students have implemented and installed the package and launch file. They are not a claim that this assignment document contains a runnable solution.

```
source /opt/ros/jazzy/setup.bash
cd smart_hub_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
ros2 launch smart_hub hub.launch.py scenario:=S8 seed:=42
ros2 node list
ros2 topic list -t
ros2 topic info /hub/queue --verbose
ros2 topic hz /hub/queue
ros2 service list -t
ros2 action list -t
ros2 interface show smart_hub_interfaces/action/EscortPassenger
ros2 service call /route/compute \
  smart_hub_interfaces/srv/ComputeRoute \
  "{start_node: 'R', goal_node: 'D', agent_type: 'robot', excluded_edges: []}"
ros2 action send_goal /robot/escort \
  smart_hub_interfaces/action/EscortPassenger \
  "{request_id: 'demo-01', passenger_id: 'P01', pickup_node: 'R', destination_node: 'D',
    max_duration_sec: 240.0}" \
  --feedback
```

Launch must accept scenario, seed, and hardware/simulation mode. Use a team-specific `ROS_DOMAIN_ID` to prevent neighboring groups from receiving each other's commands. Make hardware mode an explicit opt-in, with startup motion disabled.

20.2 Evidence capture and replay

```
ros2 bag record -o bags/S8_seed42 \
  /hub/queue /hub/demand /hub/vehicles /hub/bus_arrival \
  /parking/state /traffic/edges /crossing/state \
  /robot/odom /robot/cmd_vel_candidate /robot/cmd_vel \
  /bus/cmd_vel_candidate /bus/cmd_vel /hub/estop \
  /hub/safety /hub/events /tf /tf_static
colcon test --packages-select smart_hub
colcon test-result --verbose
```

Log action goal, acceptance, feedback summaries, cancellation, and results to `/hub/events`; the topic list above does not automatically capture every service/action transaction. Include `/clock` when simulation time is used, the relevant raw sensor stream when diagnosing perception, and a test-only ground-truth stream for evaluation. Check recorded message counts and QoS compatibility.

Replay recorded inputs into an isolated analysis namespace with command output disconnected. Replaying recorded output topics is evidence inspection, not a closed-loop rerun. For a reproducible closed-loop test, reset the simulator and rerun the seeded scenario with controllers active. Never replay historical motor commands onto hardware.

21 Verification matrix and metric definitions

21.1 Automated test layers

Use `pytest` for pure algorithms and `launch_testing` for the ROS graph. Unit tests must cover polygon membership, stale-time logic, feature extraction, reservation idempotency, edge costs, shortest paths, and state-machine transitions without requiring RViz. Integration tests launch real nodes and assert topic timing, service results, action feedback/cancellation, QoS compatibility, and final zero commands. Scenario replay tests use fixed YAML and seeds; bag replay is for analysis unless outputs and actuators are isolated. Run static checks through `ament_flake8` and `ament_pep257`, and make the submitted `colcontest-result--verbose` free of unexpected failures.

Test ID	Requirement	Stimulus	Pass / evidence
T01	S1 counting	Known 0,3,5,4 sequence	Correct stable counts; timestamped plot
T02	S1 occlusion	0.3 s missing track, repeated frames	No double count; frozen feed invalid in 1.2 s
T03	S2 prediction	Unseen episode seeds	MAE/RMSE and persistence comparison; no data leakage
T04	S2 dispatch	Threshold noise, stale forecast	One outstanding goal; explicit fallback
T05	S3 docking	Ten normal approaches	10/10; pose tolerance and stopped-door interlock
T06	S3 cancellation	Cancel during approach	Correct status; zero command within 0.5 s after acceptance
T07	S4 atomicity	Twenty concurrent request pairs	Zero double allocations; same-ID retry stable
T08	S4 expiry	Occupied/stale slot at TTL	Never falsely free; invalid release rejected
T09	S5 congestion	Free flow, slow dense flow, obstruction	Correct labels; bounded delay; no stale-as-clear error
T10	S5 stale state	Freeze and restore vehicle observations	UNKNOWN within threshold; clean recovery
T11	S6 routing	Twenty graph-state seeds	Cost matches independent oracle; no forbidden edge
T12	S6 no route	Two critical edges blocked	Explicit NO_ROUTE; zero candidate motion
T13	S7 escort	Ten nominal missions	10/10; arrival error within 0.20 m
T14	S7 goal handling	Busy, no-ready, cancel, stale odometry	Rejection or bounded terminal result; no silent hang
T15	S8 integration	Ten rush-hour seeds	Zero collision/resource conflicts; paired metrics
T16	S9 stop latch	Assert, clear input, reset attempts	Zero command in 0.20 s; deliberate restart
T17	S9 watchdog	Kill controller, then supervisor	Safety and adapter independently stop motion
T18	S9 service loss	Planner absent/late reply	Timeout, bounded retry, no stale-path execution

21.2 Metrics and measurement discipline

$$\mathrm{MAE} = \frac{1}{N}\sum_i |y_i - \widehat{y}_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_i (y_i - \widehat{y}_i)^2}, \qquad \mathrm{SuccessRate} = \frac{\text{successful accepted goals}}{\text{accepted goals}}.$$

Report rejected goals separately, with reasons. Count collision events using footprint overlap with ground-truth geometry; count crossing conflicts using simultaneous physical occupancy, not merely matching software grants. Queue wait is entry timestamp to boarding timestamp; censored passengers still waiting at run end must be reported, not dropped silently.

Use monotonic time for same-host latency. If distributed hosts are used, document clock synchronization and do not subtract unsynchronized clocks. Stop-command latency is supervisor receipt of stop to first zero output; physical stop latency ends when measured velocity is below the stopped threshold. End-to-end sensor latency begins at acquisition, not at detector publication.

For each run record: run ID, scenario ID, seed, commit hash, configuration hash, model ID, start/end time, machine specification, accepted/rejected goals, metrics, and failure reasons. Publish individual run values and aggregate mean, standard deviation, p50/p95 where sample size permits. Twenty service pairs provide limited tail evidence; report sample count with every percentile.

21.3 Required metric thresholds

Metric	Baseline acceptance threshold	Measurement source
Queue counting accuracy	Stable synthetic counts exact; video count MAE $\leq$ 1 person	Annotated frames and QueueState
Demand prediction error	Test MAE no worse than persistence; inference p95 $<$ 100 ms	Held-out episodes and model timing
Route computation time	p95 $<$ 250 ms; oracle cost match within 10^{-6} relative tolerance	Service timestamps and independent oracle
Parking reservation success	100% correct when capacity exists; zero double allocation in 20 paired trials	Service/event correlation IDs
Safety stop latency	Supervisor receipt to zero command $\leq$ 0.20 s in simulation	Safety input and final command stamps
Crossing conflict violations	Zero simultaneous incompatible occupancy or overlapping ownership	Ground truth and CrossingState
Robot mission success	10/10 nominal; final error $\leq$ 0.20 m	Action results and odometry
Bus docking accuracy	10/10 nominal; $\leq$ 0.20 m and $\leq$ 0.17 rad after one-second hold	Vehicle state and DockBus result
Safety interventions	Every intervention has a reason and correlation ID; no silent motion continuation	SafetyState and HubEvent

22 Optional physical implementation

The assessed baseline is simulation-first. A team may reproduce selected scenarios on a 3 m × 2 m tabletop at scale 1:4 only after simulation stop, watchdog, and crossing tests pass. Use a low-speed mobile robot, overhead or fixed camera, three parking sensors, red/amber/green LEDs, a servo barrier, and a hardwired emergency-stop button that removes motor power independently of ROS.

Simulation contract	Physical binding through `actuator_bridge`	Required physical evidence
`/camera/image_raw`	Calibrated USB/RGB camera with unchanged Image contract	Calibration image, timestamp rate, queue test
`/parking/observed`	Ultrasonic, infrared, or magnetic slot sensors	Empty/occupied confusion matrix and stale test
`/robot/cmd_vel`	Mobile-base driver after speed and unit conversion	Measured speed, stop time, stopping distance
`/bus/cmd_vel`	Model vehicle driver or safe simulated bus	Driver timeout and zero-command acknowledgment
Barrier request/status	Servo command plus independent entrance/occupancy interlock	Command/status log; no closure on occupied lane
Parking and safety state	LEDs: green FREE, amber RESERVED, red OCCUPIED/STOP, blinking UNKNOWN	Video synchronized with topic/event log
`/hub/estop`	Debounced button state in ROS plus independent motor-power cut	Five stop trials; software and physical latency

The bridge is the only hardware-specific layer. Core nodes keep the same topics, services, actions, frames, and units. On startup and loss of ROS communication, propulsion remains disabled, the barrier holds its declared non-trapping state, and LEDs indicate UNKNOWN/STOP. Physical results must report calibrated scale, measured deceleration, network arrangement, battery condition, and every parameter changed from simulation.

23 Seven-week work plan and five-person responsibilities

23.1 Deliverable-driven schedule

Week	Required work	Exit evidence
1	Freeze map, interfaces, ownership and safety policy; create packages and basic simulator	Map file, ROS graph, build log, queue topic, zero-motion startup
2	Implement queue perception and parking state/services; add unit tests	S1 deterministic counts; S4 reservation concurrency proof
3	Implement forecast baseline/model and bus docking	Held-out data split; S2 comparison; S3 action feedback/cancel
4	Implement congestion monitor, graph planner, and robot escort	S5 edge labels; oracle-tested S6 routes; S7 nominal/negative tests
5	Integrate crossing arbiter, safety, dashboard, and all nodes	S8 full pipeline; first S9 fault-injection report
6	Repeat experiments; improve weak cases; add physical adapter if available	T01–T18 matrix; reproducible bags; measured hardware limits
7	Freeze code; record final demonstration; submit and defend	Report, source, models/data, reproducible run instructions, individual defense

23.2 Five members: ownership and review

Member	Primary tasks	Independent review / demonstration
M1	Detector, tracking, demand features, model training, S1/S2 experiments	Review M3 state validity; explain leakage prevention
M2	Coordinator, bus manager, dispatch hysteresis, crossing arbitration, S3/S8	Review M4 cancellation; explain resource ownership
M3	Parking manager, idempotency, traffic monitor, S4 concurrency tests	Review M1 data labels; explain atomic updates
M4	Map graph, route planner, robot manager, S6/S7 tests	Review M2 crossing logic; demonstrate blocked-route recovery
M5	Simulator, traffic monitor, safety, launch, actuator adapter, S5/S9	Review integrated failures; demonstrate watchdog and reset

Every member writes tests for their own subsystem and reviews another subsystem. M5 coordinates integration tooling; all members remain responsible for fixing their own integration defects. Keep weekly issues with owner, estimate, acceptance evidence, and reviewer.

23.3 If the class has up to five teams

Each team builds the same complete baseline so assessment remains comparable. Optional emphasis: Team 1 passenger perception; Team 2 transit prediction; Team 3 parking coordination; Team 4 autonomous assistance; Team 5 traffic optimization. Extensions start only after the integrated baseline passes. Do not split the class into five dependent subsystems unless the instructor explicitly changes the assessment: one absent team must not prevent others from demonstrating their system.

24 Submission, grading, demonstration, and references

24.1 Submission package

1. **Source repository snapshot:** Python packages, interface definitions, launch files, parameters, map, scenario files, tests, dependency versions, license notices, and commit hash.
2. **Engineering report, suggested 12–18 pages excluding appendices:** requirements, coordinate map, ROS graph, interface rationale, state machines, prediction/data split, safety policy, all T01–T18 results, limitations, and contribution table.
3. **Evidence directory:** annotated videos, representative rosbag recordings, full metric CSVs, training/evaluation scripts, model artifact, and sufficient input data or lawful download instructions to reproduce results.
4. **Eight-minute demonstration:** 1 minute map/architecture; 2 minutes nominal detection/dispatch/-docking/parking; 2 minutes robot and rerouting; 2 minutes faults/cancel/reset; 1 minute metrics and limitations.
5. **Individual defense:** each student explains one publisher/subscriber path, one transaction or action lifecycle, and one failed test they corrected.

24.2 Assessment rubric

Category	Points	Full-credit evidence
ROS 2 architecture	10	Typed contracts, justified topic/service/action/parameter use, QoS, frames, and clean ownership
Scenario implementation	14	All eight functional scenarios operate with specified success and failure behavior
Testing quality	10	Complete T01–T18 matrix, unit/integration tests, bags, repeated runs, and traceable metrics
Safety behavior	10	Independent watchdogs, interlocks, stop latch, crossing exclusion, and deliberate reset
AI prediction	8	Valid data split, persistence baseline, held-out evaluation, latency, and safe fallback
Routing and coordination	10	Correct edge states, optimal valid routes, replanning, atomic resources, and action coordination
Code quality	6	Readable Python, bounded callbacks, errors handled, package metadata, and automated checks
Documentation	6	README, interface rationale, diagrams, parameter record, limitations, and reproducible instructions
Demo performance	6	Coherent live/replayed run, visible physical or simulated response, and clear evidence
Individual contribution	20	Traceable implementation, cross-review work, technical defense, and repository evidence
Total	100	Team 80 + individual 20

A failed test must remain in the report with a diagnosis; deleting evidence does not convert failure into success. Physical testing proceeds only after stop and watchdog checks pass. Simulation evidence remains the baseline when hardware is unavailable.

24.3 Technical references

ROS APIs and communication concepts follow the official Jazzy documentation. Map dimensions, thresholds, tests, policies, and grading weights are assignment design choices. Full-page retrieval of some documentation was access-restricted during preparation; verify API details against the installed distribution. No ROS runtime execution of the starter fragments is claimed.

[1] ROS 2: Writing a simple service and client (Python).
[2] ROS 2: Writing an action server and client (Python).
[3] ROS 2: Quality of Service settings.
[4] ROS 2: Creating an action.
[5] ROS 2: rosbag2 recording and playback QoS overrides.